

Transacciones

Del COMMIT local a la Saga distribuida.



AGENDA

El recorrido de hoy

01

Repaso · 15 min

Transacciones y ACID

Qué es una transacción y qué promete cada letra.

02

Repaso · 25 min

Concurrencia en una BD

Anomalías, niveles de aislamiento y serializable.

03

30 min

El mundo distribuido

Varias bases de datos: commit en dos fases (2PC).

04

45 min

Sagas y Outbox

Compensaciones, orquestación y coreografía.

05

20 min

Circuit breaker

Que un servicio lento no tumbe a los demás.

06

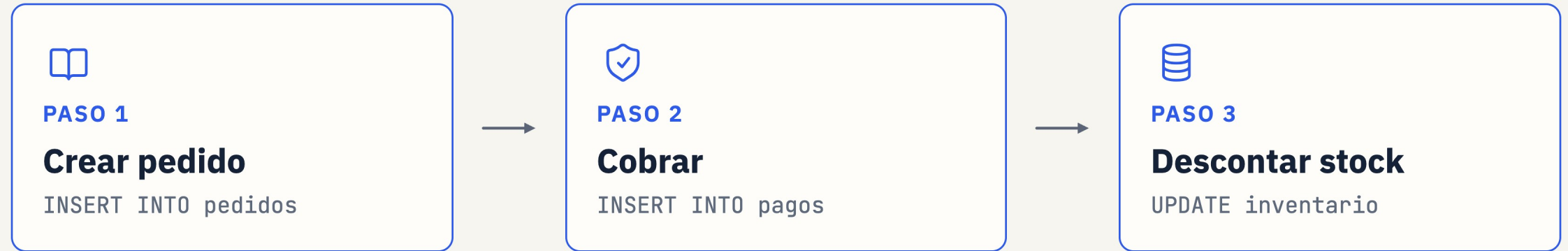
20 min

Cierre y actividad

Puente a mensajería y diseño de una saga en equipos.

Hilo conductor: una tienda en línea que crece de monolito a microservicios. Receso de 10 min después del bloque 03.

Comprar en línea son tres pasos



¿Y si el cobro sale bien... pero el inventario falla?



Cliente pagó sin producto

Stock que no cuadra

¿Reintento = doble cobro?

01

Transacciones y ACID

Repaso rápido de Bases de Datos: qué es una transacción y qué promete cada letra.

Una transacción es todo o nada

Un grupo de lecturas y escrituras que la base de datos trata como **una sola unidad**: o se confirman todas (**COMMIT**) o no queda ninguna (**ROLLBACK**).

```
BEGIN;  
  INSERT INTO pedidos VALUES (42, 'ana');  
  INSERT INTO pagos VALUES (42, 500);  
  UPDATE inventario SET stock = stock - 1;  
COMMIT; -- o ROLLBACK;
```

La promesa para quien programa: **si algo falla, solo hay que reintentar.**



Cuatro letras, cuatro promesas

A

Atomicidad

Todo o nada: si algo falla a la mitad, se deshace lo que ya se hizo.

En la tienda: no queda un pedido sin su cobro.

C

Consistencia

Las reglas del negocio (invariantes) se cumplen antes y después.

stock ≥ 0 . Ojo: la define tu aplicación, no la BD.

I

Aislamiento

Transacciones simultáneas no se pisan: cada una actúa como si estuviera sola.

En la tienda: dos clientes compran el último artículo.

D

Durabilidad

Lo confirmado sobrevive a un apagón o a un crash.

Log (WAL) + fsync a disco, y réplicas.

Si falla a la mitad, se deshace

En ACID, atomicidad no habla de hilos: habla de **poder abortar**. Si una escritura falla, las anteriores de la misma transacción se descartan.



02

Concurrencia en una sola base de datos

Lo que ya vieron en Sistemas Concurrentes, aplicado a la BD: qué sale mal cuando muchos escriben a la vez.

Seis formas de pisarse

Solo hay riesgo cuando dos transacciones tocan **los mismos datos** y al menos una **escribe**.

Lectura sucia

dirty read

Ves un pedido que otra transacción aún no confirma... y que luego se aborta.

Escritura sucia

dirty write

Dos compras se mezclan: el pedido queda a nombre de uno y el cobro del otro.

Lectura no repetible

read skew

Sumas tus dos cuentas en plena transferencia y «faltan» \$100.

Actualización perdida

lost update

Dos ventas leen stock = 5 y las dos escriben 4.

Sesgo de escritura

write skew

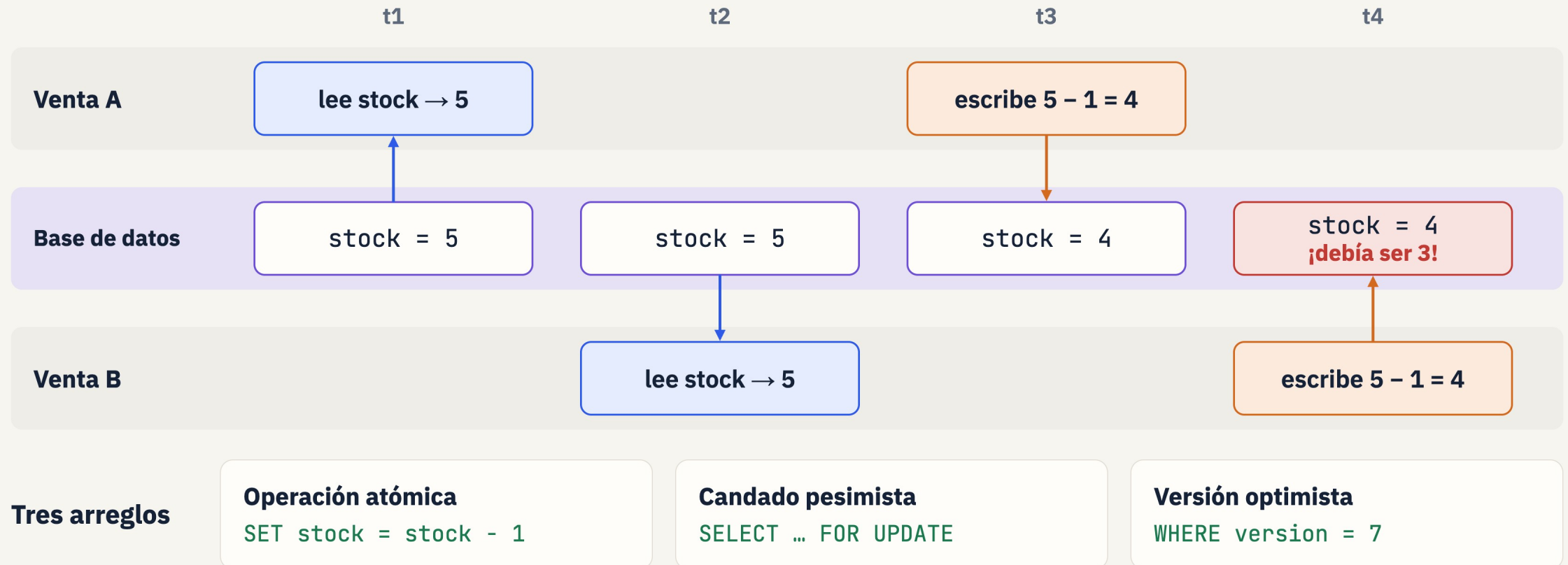
Dos médicos de guardia se dan de baja a la vez: no queda ninguno.

Fantasma

phantom

Revisas que la sala esté libre y otro la reserva entre tu SELECT y tu INSERT.

Leer, restar, escribir... y perder



Cada quien tuvo razón; juntos, no

Transacción de Ana

1 · ¿Cuántos de guardia?

`SELECT count(*) → 2`



2 · $2 \geq 2$: puedo irme

la regla se cumple... hoy



3 · Ana sale · COMMIT

`UPDATE ... fila de Ana`

Tabla de guardias

Antes	De guardia
Ana	sí
Beto	sí



Después	De guardia
Ana	no
Beto	no

0 médicos de guardia

Transacción de Beto

1 · ¿Cuántos de guardia?

`SELECT count(*) → 2`



2 · $2 \geq 2$: puedo irme

su foto aún dice 2



3 · Beto sale · COMMIT

`UPDATE ... fila de Beto`

No es lost update: cada quien escribió **su propia fila**. Snapshot isolation no lo evita; hace falta **serializable** o bloquear lo leído con FOR UPDATE.

Más aislamiento, menos sorpresas

Nivel	Lectura sucia	Read skew	Lost update	Write skew y fantasmas	Dónde lo encuentras
Read uncommitted	Posible	Posible	Posible	Posible	Casi nadie lo usa
Read committed	Evitado	Posible	Posible	Posible	Default de PostgreSQL, Oracle y SQL Server
Snapshot isolation	Evitado	Evitado	Depende	Posible	«Repeatable read» en PostgreSQL; «serializable» en Oracle
Serializable	Evitado	Evitado	Evitado	Evitado	PostgreSQL (SSI); MySQL y SQL Server (2PL)



El nombre no garantiza el comportamiento: «repeatable read» significa cosas distintas en PostgreSQL, MySQL y Db2.
Revisa qué hace tu motor.

Tres caminos a serializable

Pesimista extremo

Ejecución en serie

Un solo hilo; una transacción a la vez.

Pro: sin candados ni conflictos.

Contra: un núcleo; transacciones cortas y en memoria.

Redis · VoltDB · Datomic

Pesimista

2PL · two-phase locking

Candados para leer y para escribir, retenidos hasta el COMMIT.

Pro: evita todas las carreras.

Contra: lectores y escritores se bloquean; deadlocks.

MySQL · SQL Server

Optimista

SSI · snapshot serializable

Lee de una foto; al COMMIT revisa conflictos y aborta si hace falta.

Pro: nadie espera a nadie.

Contra: con mucha contención, muchos abortos.

PostgreSQL · CockroachDB

Con SERIALIZABLE, **tu código debe reintentar** la transacción abortada (en PostgreSQL, error 40001).

ANTES DE SALIR DE LA BASE DE DATOS

Todo esto funciona porque hay un solo disco que decide.

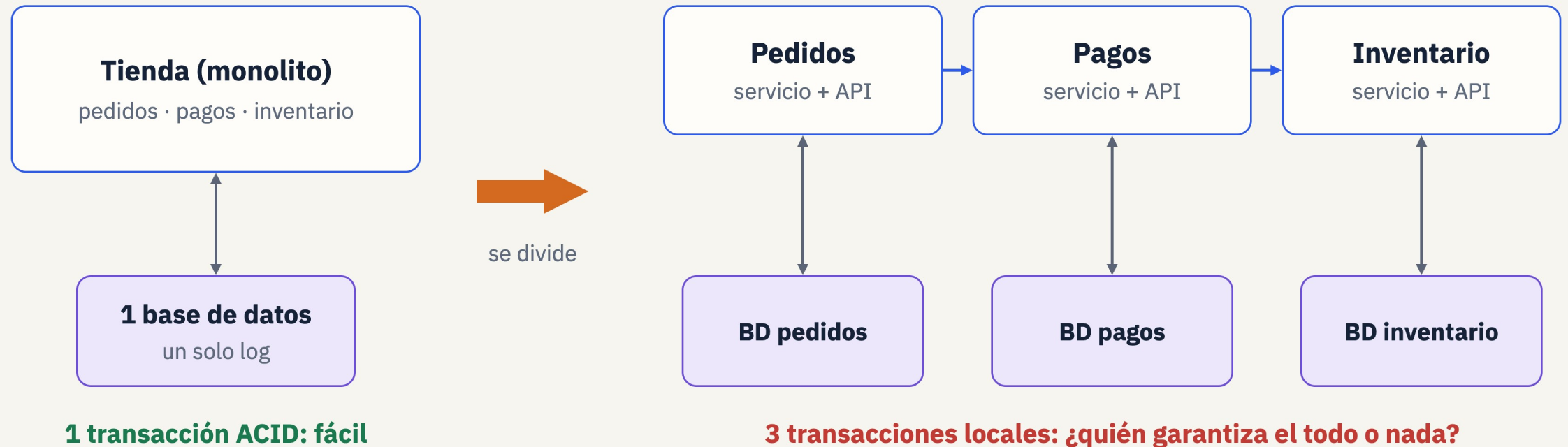
En una base de datos, el COMMIT es atómico gracias a **un único registro en el log**.
¿Qué pasa cuando la tienda tiene tres bases de datos?

03

El mundo distribuido

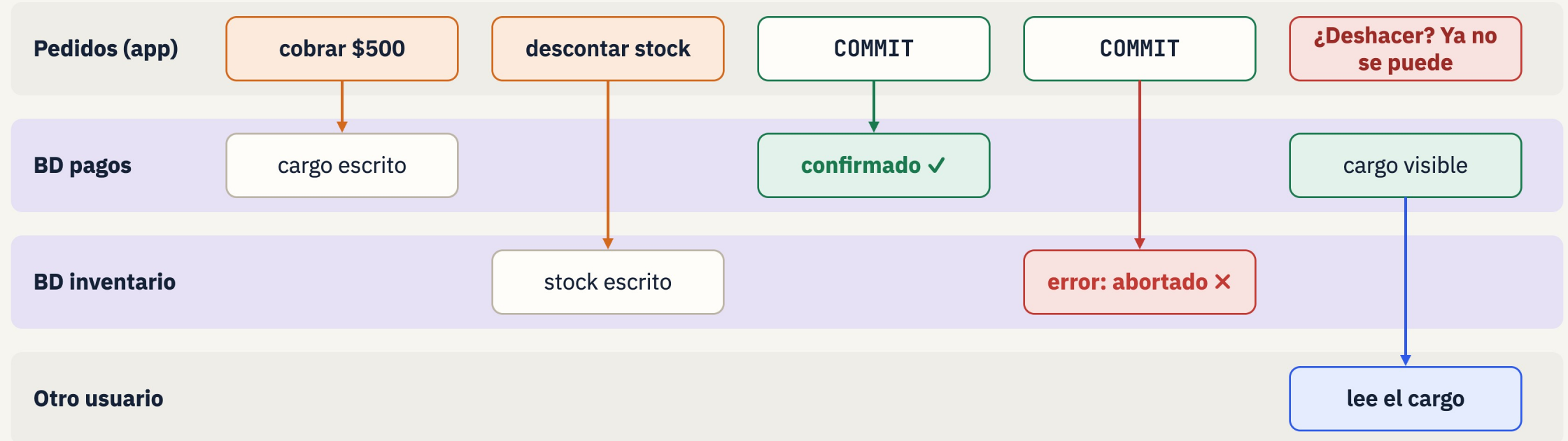
Partimos la tienda en servicios... y la atomicidad tiene que cruzar la red: commit en dos fases (2PC).

Un monolito, luego tres bases de datos



03 · EL PROBLEMA

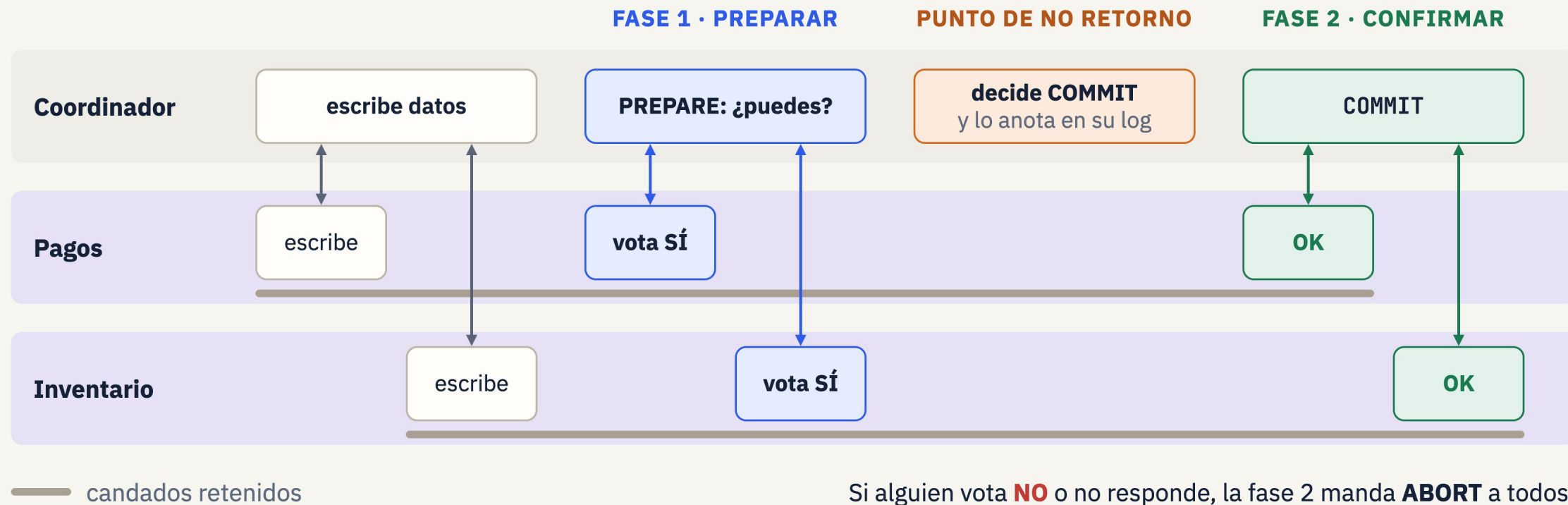
Commit aquí, error allá



Lo confirmado ya es visible: no se puede «desconfirmar». Necesitamos que **todos confirmen o nadie**: el problema del **commit atómico**.

03 · TWO-PHASE COMMIT (2PC)

Primero se pregunta; luego se decide



Dos promesas que no se rompen



1 · El participante vota SÍ

Ya escribió todo a disco y revisó sus restricciones. **Renuncia a abortar por su cuenta:** si le piden COMMIT, lo hará aunque se haya reiniciado.



2 · El coordinador decide

Anota la decisión en su log **antes** de avisar. Es irrevocable: reintenta COMMIT para siempre hasta que todos respondan.



Como una boda: antes del «acepto» puedes decir que no; **después, ya no**. Si te desmayas justo tras decirlo, igual quedaste casado.

En una sola base de datos, estas dos promesas son un solo evento: escribir el registro de commit.

03 · EL PUNTO DÉBIL

Si el coordinador cae, todos esperan



Depende de quién coordine

Entre tecnologías distintas

XA · JTA · PostgreSQL + broker

- El coordinador vive dentro de tu app: si muere, los participantes quedan en duda.
- Candados retenidos minutos... o para siempre.
- Mínimo común denominador: sin detección de deadlocks ni SSI entre sistemas.

Evítalo si puedes

Dentro de una BD distribuida

Spanner · CockroachDB · DynamoDB

- Coordinador replicado con consenso (Raft o Paxos): si cae, otro sigue.
- Coordinador y shards hablan directo, sin la app en medio.
- DynamoDB guarda cada transacción en un ledger; un recovery manager retoma las atoradas.

Funciona bien

2PC no es malo: es frágil cuando el coordinador es **un punto único de falla**.

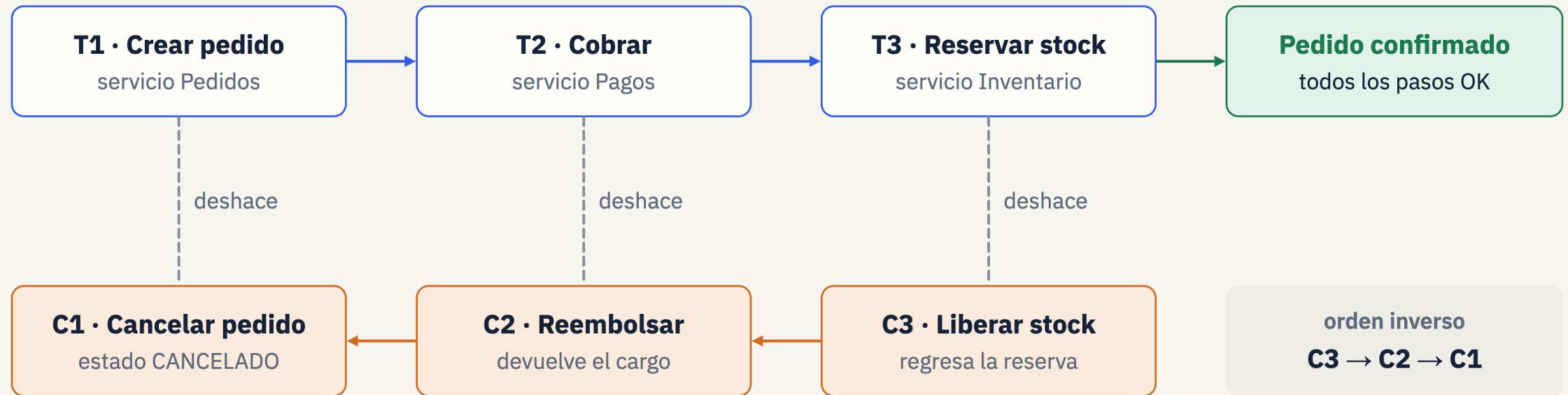
04

Sagas y Outbox

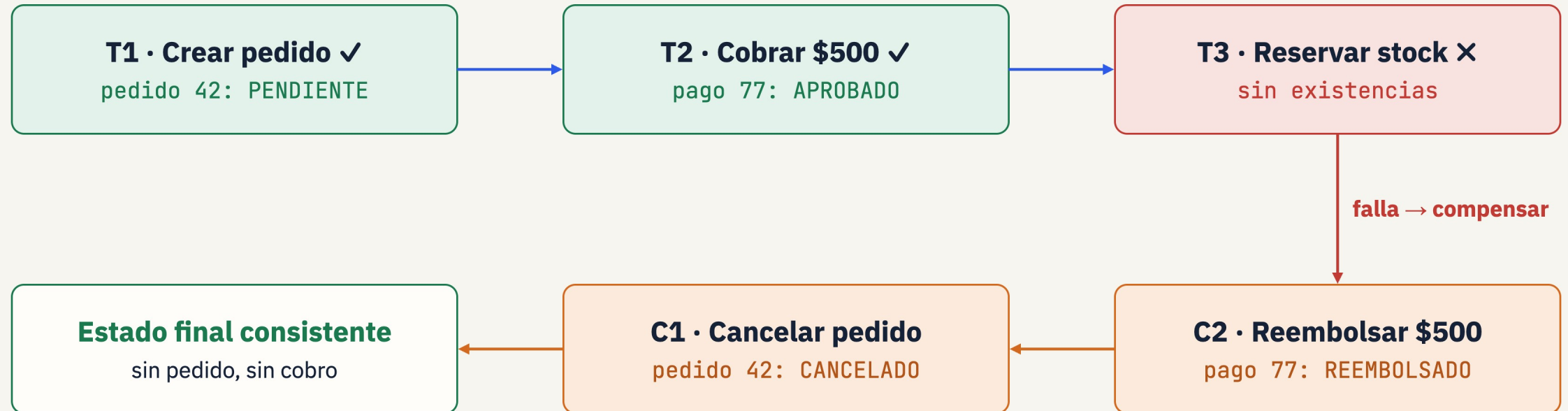
Si no podemos bloquear a todos, avanzamos paso a paso... y planeamos cómo regresar.

Pasos locales, regresos planeados

Una **saga** es una secuencia de transacciones locales $T1...Tn$. Cada Ti tiene una **compensación** Ci que deshace su efecto de negocio. Si un paso falla, las compensaciones corren en orden inverso.



Falla el inventario: se compensa



Por unos segundos el cliente vio el cobro: el sistema fue **inconsistente un momento** y luego se corrigió. Eso es **consistencia eventual**.

Lo que la saga no te da

Una saga da atomicidad *eventual*, consistencia y durabilidad... pero **no da aislamiento**.

Lo que pierdes

Aislamiento: otros ven estados intermedios, como un pedido cobrado que aún puede cancelarse.

Rollback real: compensar no es deshacer; un correo enviado no se «desenvía».

Simplicidad: cada paso necesita su compensación, y la compensación también puede fallar.

Cómo se defiende

Estados PENDIENTE: marcan lo que aún no es definitivo.

Idempotencia: pasos y compensaciones se repiten sin duplicar efectos.

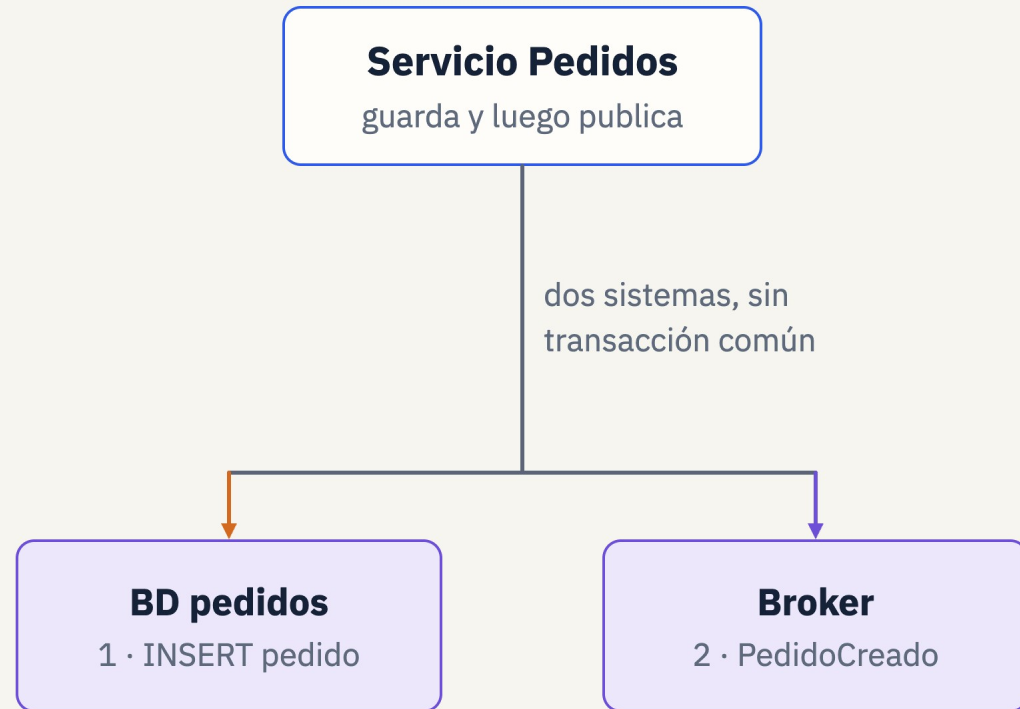
Orden: lo reversible primero; lo irreversible (correo, envío) al final.

Estado persistido: si el coordinador cae, otro retoma la saga.

2PC contra Saga

Criterio	2PC	Saga
Atomicidad	Estricta: todos o ninguno	Eventual, vía compensaciones
Aislamiento	Sí, con candados	No: estados intermedios visibles
Bloqueo	Candados hasta el COMMIT; en duda si cae el coordinador	No bloquea entre servicios
Disponibilidad	Baja: todos deben responder	Alta: cada paso avanza solo
Complejidad	En la infraestructura	En tu código: compensaciones
Úsalo para	Shards de una misma BD distribuida	Microservicios con BD propias

Guardar y avisar son dos escrituras



BD sí, broker no

El pedido existe, pero nadie se entera: no se cobra ni se reserva.

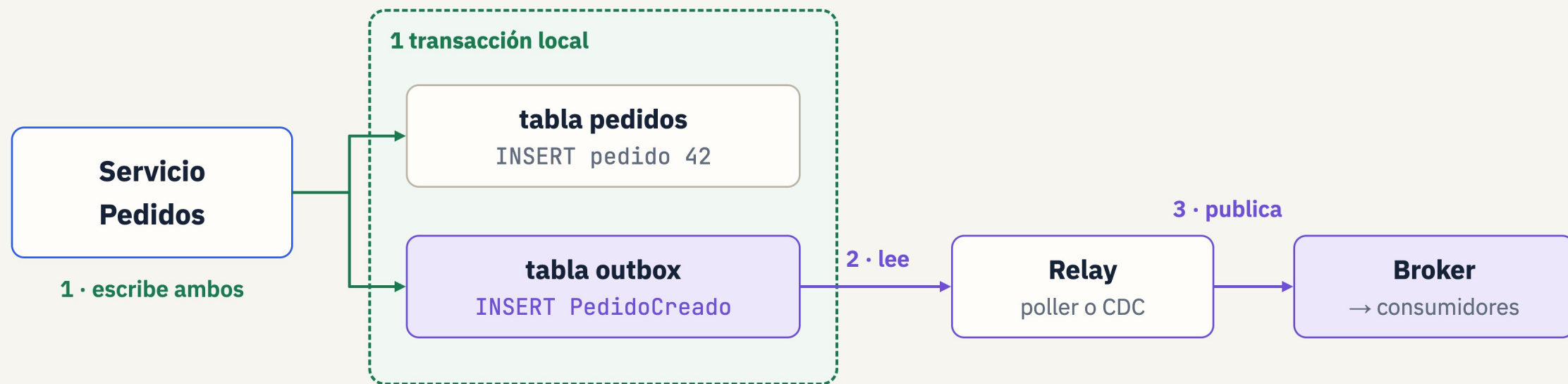
Broker sí, BD no

Llega un evento de un pedido que no existe.

Se cae entre las dos

No sabes cuál se hizo. ¿Reintentar duplica el evento?

Escribe una vez; publica después



Atómico: el pedido y su evento se guardan juntos o ninguno.

Al menos una vez: el relay puede publicar un evento dos veces.

Por eso: el consumidor guarda los IDs ya procesados.

Un director da las órdenes



A favor: el flujo está escrito en un solo lugar; es fácil de leer, probar y depurar.

En contra: concentra la lógica y debe guardar su estado para sobrevivir caídas.

Un orquestador en 18 líneas

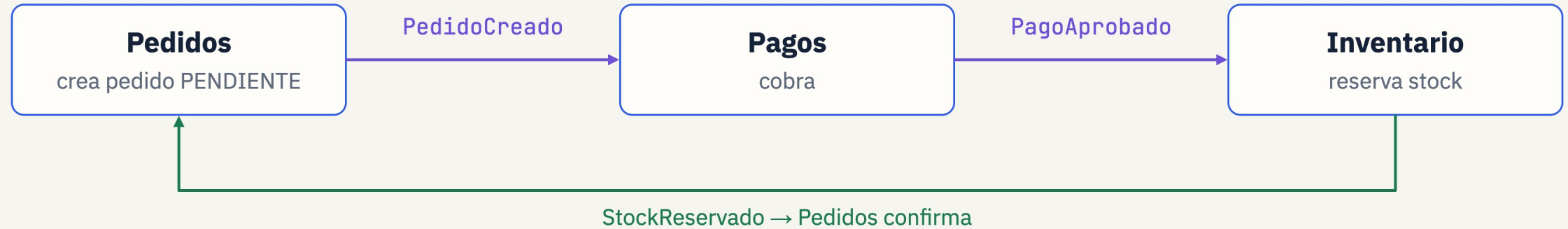
```
interface Paso {
    void ejecutar(Saga s);    // lanza excepción si falla
    void compensar(Saga s);  // idempotente
}

void ejecutarSaga(Saga s, List<Paso> pasos) {
    Deque<Paso> hechos = new ArrayDeque<>();
    for (Paso paso : pasos) {
        try {
            paso.ejecutar(s);
            hechos.push(paso);           // pila: LIFO
            sagas.guardar(s.avanzo(paso)); // estado persistido
        } catch (RuntimeException e) {
            while (!hechos.isEmpty()) hechos.pop().compensar(s);
            sagas.guardar(s.fallida()); return;
        }
    }
    sagas.guardar(s.completada());
}
```

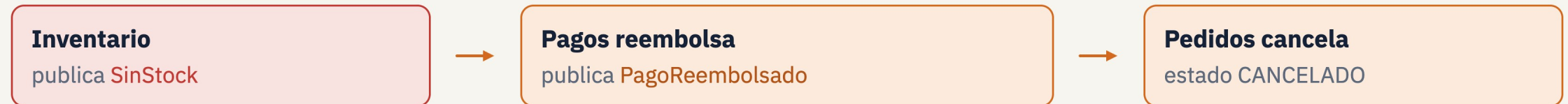
- 1 Cada paso trae su **compensación**.
- 2 La pila (pop) compensa en **orden inverso** sin esfuerzo.
- 3 **Persiste el estado**: si el proceso muere, otro retoma la saga.
- 4 Usa s.id() + paso como llave para que cada paso sea **idempotente**.

Bug clásico: un catch que solo hace log.error(e) y sigue: la saga avanza como si nada y nunca compensa.

Nadie manda; todos escuchan



Si no hay stock, la compensación también viaja por eventos:



A favor: desacoplado; cada servicio solo conoce eventos.

En contra: nadie ve el flujo completo; cuesta seguirlo.

Escucha, transacción local, publica

```
// Servicio de Pagos
@KafkaListener(topics = "pedido-creado")
@Transactional // marca + cobro + outbox
void onPedidoCreado(PedidoCreado e) {
    if (!procesados.marcar(e.id())) return;
    var id = e.pedidoId();
    outbox.publicar(pagos.cobrar(id, e.total())
        ? new PagoAprobado(id) : new PagoRechazado(id));
}

@KafkaListener(topics = "sin-stock")
@Transactional
void onSinStock(SinStock e) {
    if (!procesados.marcar(e.id())) return;
    pagos.reembolsar(e.pedidoId());
    outbox.publicar(new PagoReembolsado(e.pedidoId()));
}
```

```
// Servicio de Inventario
@KafkaListener(topics = "pago-aprobado")
void onPagoAprobado(PagoAprobado e) {
    if (!procesados.marcar(e.id())) return;
    var id = e.pedidoId();
    outbox.publicar(inventario.reservar(id)
        ? new StockReservado(id) : new SinStock(id));
}
```

```
// Servicio de Pedidos
@KafkaListener(topics = "resultado-saga")
void on(ResultadoSaga e) {
    switch (e) {
        case StockReservado s → confirmar(s.pedidoId());
        case PagoRechazado r → cancelar(r.pedidoId());
        case PagoReembolsado r → cancelar(r.pedidoId());
    }
}
```

Director u orquesta de jazz

Criterio	Orquestación	Coreografía
Quién conoce el flujo	El orquestador	Nadie: está repartido
Acoplamiento	Todos dependen del orquestador	Solo de los eventos
Seguir y depurar	Fácil: un solo lugar	Difícil: hace falta rastreo distribuido
Punto único de falla	Sí, si no persiste ni se replica	No
Riesgo típico	Orquestador «dios» con toda la lógica	Ciclos y espagueti de eventos
Ideal para	Flujos largos o con reglas complejas	Flujos cortos, de 2 a 4 servicios



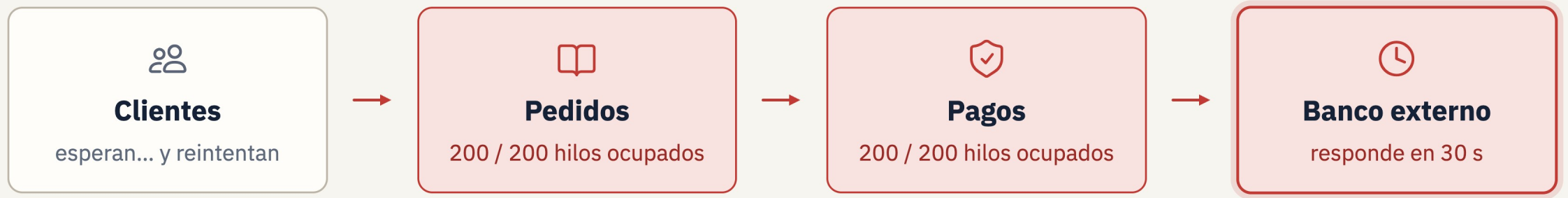
Regla práctica: pocos pasos y equipos independientes, coreografía; muchos pasos o reglas de negocio, orquestación. Se pueden combinar.

05

Resiliencia: circuit breaker

La saga asume que los servicios responden. ¿Y si uno no se cae, sino que se pone lento?

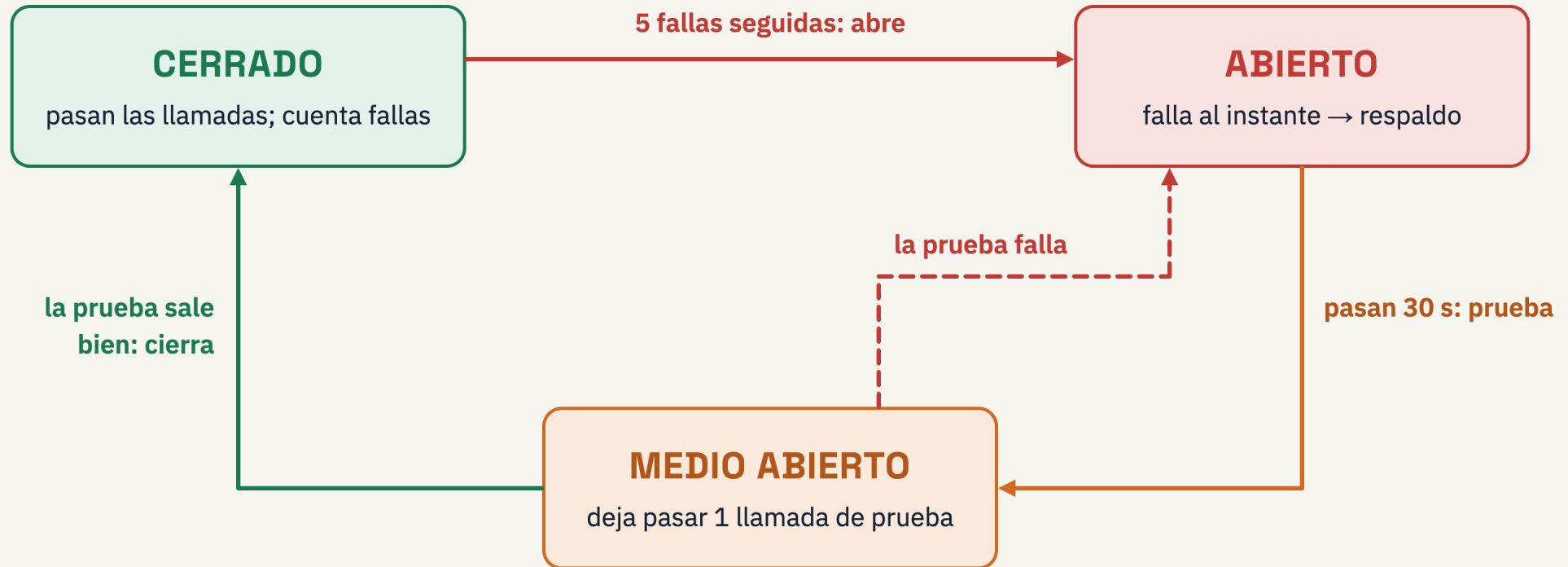
Un servicio lento tumba a todos



- 1** El banco no está caído: solo **tarda 30 s** en responder.
- 2** Pagos acumula llamadas abiertas y **se queda sin hilos** ni conexiones.
- 3** Pedidos espera a Pagos: **la tienda entera** deja de responder.

Esperar para siempre es peor que fallar rápido.

Cerrado, abierto, medio abierto



Como el interruptor de tu casa: **corta antes de que se queme todo** y, después de un rato, vuelve a intentar.

Un interruptor en 19 líneas

```
class CircuitBreaker {
    enum Estado { CERRADO, ABIERTO, MEDIO_ABIERTO }
    Estado estado = CERRADO; int fallas; Instant desde;
    final int umbral = 5; final Duration espera = ofSeconds(30);
    <T> T llamar(Supplier<T> fn, Supplier<T> respaldo) {
        if (estado == ABIERTO) {
            if (now().isBefore(desde.plus(espera))) return respaldo.get();
            estado = MEDIO_ABIERTO; // una prueba
        }
        try {
            T r = fn.get(); // con timeout de 2 s
            estado = CERRADO; fallas = 0; return r;
        } catch (RuntimeException e) {
            if (estado != CERRADO || ++fallas ≥ umbral)
                { estado = ABIERTO; desde = now(); }
            return respaldo.get();
        }
    }
}
```

umbral = 5

fallas seguidas para abrir

espera = 30 s

tiempo abierto antes de probar

timeout = 2 s

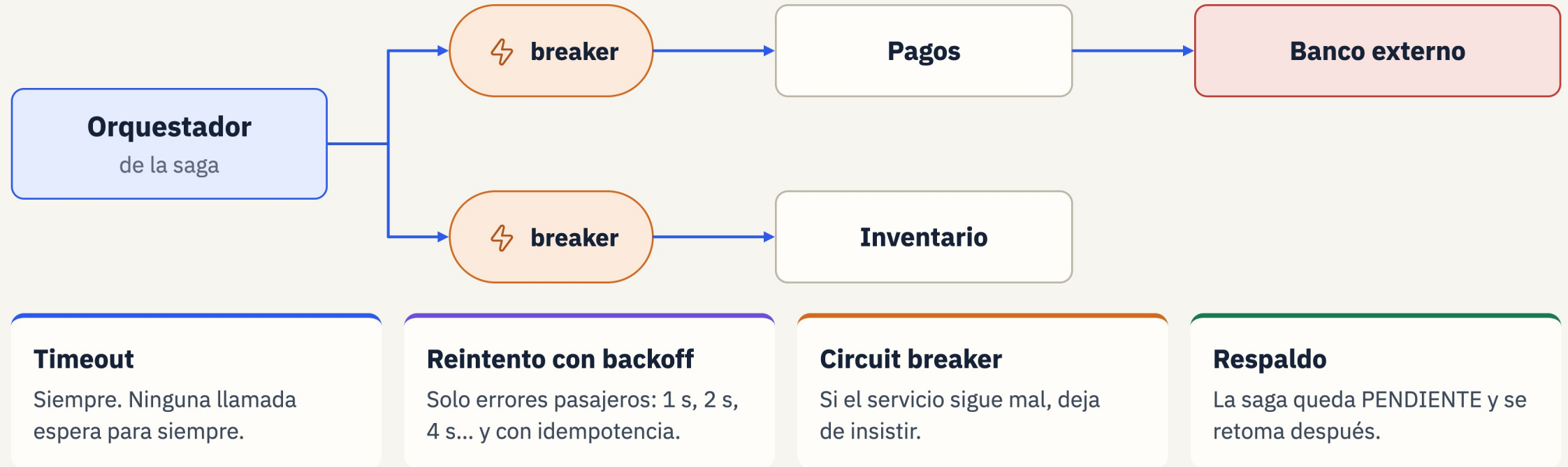
nunca esperar para siempre

respaldo

respuesta honesta: «pago en proceso»

En producción: Resilience4j y su anotación
@CircuitBreaker.

Cuatro defensas en cada llamada



Con el breaker abierto, la compra no truena: **espera en un estado conocido** hasta que el banco se recupere.

Hoy el broker fue una caja negra

Lo usamos en Outbox y en la coreografía sin abrirlo. La próxima clase respondemos:



¿Cola o tópico?

Punto a punto contra publicar y suscribir.



¿Cuántas veces llega?

A lo más una, al menos una o exactamente una.



¿Se respeta el orden?

Particiones y llaves de orden.



¿Y los que siempre fallan?

Reintentos y cola de mensajes muertos (DLQ).

CIERRE

El mapa de hoy



Mientras más se reparte el sistema, más garantías pasan de la base de datos a tu diseño.

Diseña tu propia saga



Agencia de viajes: reservar un viaje completo

Cada proveedor es un servicio con su propia base de datos.

Vuelos

Hotel

Autos

Pago

- 1 Lista los pasos y la **compensación** de cada uno.
- 2 ¿**Orquestación o coreografía**? Justifica.
- 3 ¿Qué paso **no se puede compensar** y en qué orden lo pones?
- 4 ¿Qué eventos o comandos necesitas, y dónde usarías **Outbox**?
- 5 ¿Dónde va un **circuit breaker** y qué respaldo darías?

Referencias

LIBROS Y ARTÍCULOS

Kleppmann, M. y Riccomini, C. *Designing Data-Intensive Applications*, 2.^a ed., cap. 8 «Transactions». O'Reilly.

Härder, T. y Reuter, A. (1983). «Principles of Transaction-Oriented Database Recovery». *ACM Computing Surveys*.

Garcia-Molina, H. y Salem, K. (1987). «Sagas». *ACM SIGMOD*.

Richardson, C. (2018). *Microservices Patterns*. Manning. Saga, Outbox y base de datos por servicio.

Nygard, M. (2018). *Release It!*, 2.^a ed. Pragmatic Bookshelf. Circuit breaker.

Fowler, M. (2014). «CircuitBreaker». martinfowler.com.

Idziorek, J. et al. (2023). «Distributed Transactions at Scale in Amazon DynamoDB». *USENIX ATC*.

VIDEOS COMPLEMENTARIOS

Clase 12 · Transacciones distribuidas

TA050 Sistemas Distribuidos I (FIUBA). 2PC y el caso DynamoDB.

Transacciones distribuidas

Valle Grande. Saga, Outbox y demo con Spring Boot y Kafka.

Patrón Saga

Del Código a la Arquitectura. Orquestador en TypeScript, 19 min.